Bài 8: Kiến trúc AI Agent – Planning, Memory và Tool Use

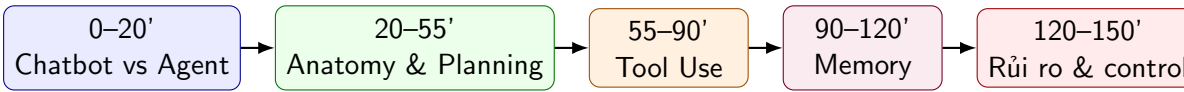
AI Engineer – Module 3: AI ứng dụng

Lecture deck – 2,5 giờ

Mục tiêu Bài 8

- Phân biệt chatbot và agent ở góc nhìn thiết kế hệ thống.
- Hiểu ba trụ cột kiến trúc agent: **Planning, Memory, Tool Use**.
- Giải thích agent loop: mục tiêu $\rightarrow$ lập kế hoạch $\rightarrow$ chọn tool $\rightarrow$ quan sát $\rightarrow$ bước tiếp theo.
- Hiểu tool calling, state/memory và quyền truy cập ở mức ứng dụng.
- Thiết kế human approval cho hành động rủi ro.

Bản đồ 2,5 giờ



Thông điệp chính

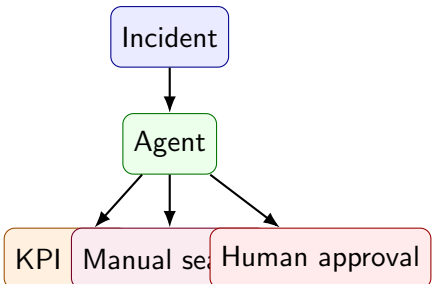
Agent không phải là “AI tự chủ hoàn toàn”; agent là LLM được đặt trong workflow có **planning, memory, tools** và kiểm soát.

Từ AI trả lời sang AI làm việc theo quy trình

- Bài 1 & 3: LLM trả lời tốt hơn khi ta giao việc rõ ràng (prompt).
- Bài 5–6: RAG giúp LLM dùng tri thức nội bộ có căn cứ.
- Bài 4 & 7: CV/NLP (và fine-tuning) giúp xử lý ảnh và text thành dữ liệu có cấu trúc.
- Bài 8: AI bắt đầu sử dụng tool và đi theo workflow nhiều bước.

Business case mở đầu: Incident Assistant

- Có một incident mạng cần phân tích nhanh.
- AI cần đọc mô tả, tra KPI, tìm hướng dẫn trong sổ tay và đề xuất bước kiểm tra.
- Một số hành động chỉ được phép sau khi người vận hành duyệt.



Chatbot là gì?

- Nhận câu hỏi hoặc yêu cầu.
- Sinh câu trả lời dựa trên prompt, context và tri thức model/RAG.
- Thường dừng ở output ngôn ngữ: giải thích, tóm tắt, trích xuất, đề xuất.
- Nếu có hành động sau đó, thường do con người tự làm bên ngoài hệ thống.

Agent là gì?

- Agent là một workflow có LLM làm bộ phận quyết định hoặc điều phối.
- Agent có thể chọn tool, gọi tool, đọc kết quả và quyết định bước kế tiếp.
- Agent có thể giữ state trong một phiên xử lý.
- Agent cần guardrail vì tool có thể tạo tác động thật.

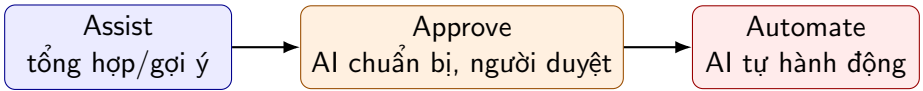
Chatbot vs Agent

Tiêu chí	Chatbot	Agent
Vai trò	Trả lời/gợi ý	Điều phối nhiều bước
Tool	Ít hoặc không có	Có gọi tool rõ ràng
State	Ngắn, theo hội thoại	Theo tiến trình xử lý
Rủi ro	Chủ yếu là thông tin sai	Sai thông tin + sai hành động
Kiểm soát	Prompt/evaluation	Permission, validation, approval

Không gắn agent với tự động hóa tuyệt đối

- Agent có thể chỉ hỗ trợ chuẩn bị thông tin.
- Agent có thể đề xuất hành động nhưng cần người duyệt.
- Agent chỉ nên tự động hóa khi hành động có rủi ro thấp, dễ kiểm tra và dễ đảo ngược.
- "Có agent" không có nghĩa là bỏ con người khỏi quy trình.

Ba mức sử dụng agent



Rủi ro và yêu cầu kiểm soát tăng dần

Ví dụ: cùng một incident, ba mức agent

- Assist: tóm tắt incident, liệt kê dữ kiện thiếu.
- Approve: đề xuất chạy kiểm tra hoặc mở ticket phụ để người duyệt.
- Automate: tự gọi tool thay đổi cấu hình chỉ khi chính sách cho phép.
- Trong Module 3, ưu tiên Assist và Approve, không khuyến khích automate sớm.

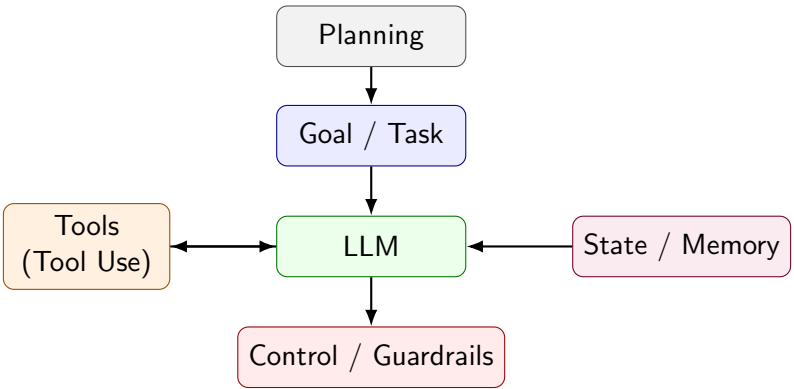
Mini activity: đây có phải agent không?

1. Chatbot trả lời câu hỏi từ tài liệu nội bộ.
2. Hệ thống đọc ticket, gọi hàm lấy KPI rồi đề xuất nguyên nhân.
3. Prompt chuyển báo cáo thành JSON.
4. Workflow tạo nháp email nhưng phải người dùng bấm gửi.

Câu hỏi

Điểm nào làm một giải pháp bắt đầu giống agent?

Agent anatomy – ba trụ cột



Ba trụ cột: **Planning** (chia và sắp bước), **Memory** (giữ trạng thái), **Tool Use** (gọi công cụ). LLM là bộ quyết định; Control bao quanh.

Thành phần 1: Goal / Task

- Goal cần cụ thể: hỗ trợ quyết định nào, cho ai, trong giới hạn nào.
- Không nên giao nhiệm vụ quá rộng như "xử lý toàn bộ incident".
- Nên tách thành các bước kiểm tra được.
- Goal tốt giúp agent không gọi tool lan man.

Thành phần 2: LLM làm gì trong agent?

- Hiểu yêu cầu và context.
- Chọn tool phù hợp hoặc quyết định không gọi tool.
- Đọc kết quả tool trả về.
- Tổng hợp kết luận hoặc đề xuất bước tiếp theo.

Thành phần 3: Tools (Tool Use)

- Tool là hàm hoặc API mà agent được phép gọi.
- Ví dụ: `get_kpi(site_id)`, `search_manual(query)`, `create_recommendation(data)`.
- Tool cần input rõ, output rõ, lỗi rõ.
- Tool càng có quyền thật thì càng cần permission và logging.

Thành phần 4: State / Memory

- State lưu tiến trình xử lý trong một phiên: dữ kiện đã biết, tool đã gọi, kết quả đã quan sát.
- Không nhầm state tạm thời với memory dài hạn.
- State giúp tránh lặp lại tool và giúp giải thích quyết định.
- State cần giới hạn để tránh rò rỉ dữ liệu giữa các phiên.

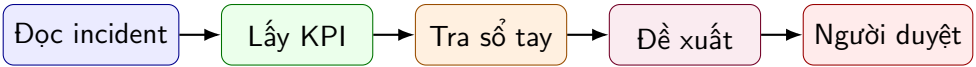
Thành phần 5: Control

- Control quyết định agent được làm gì và không được làm gì.
- Gồm: policy, allowed tools, schema validation, budget, timeout, logging, human approval.
- Control không phải phần "trang trí"; đây là phần làm agent dùng được trong nghiệp vụ.
- Không có control thì agent chỉ là demo rủi ro.

Trụ cột Planning ở mức workflow

- Planning không nhất thiết là thuật toán phức tạp.
- Với người mới, planning nên là danh sách bước nghiệp vụ có điều kiện rõ.
- Ví dụ: đọc incident $\rightarrow$ lấy KPI $\rightarrow$ tra sổ tay $\rightarrow$ đề xuất $\rightarrow$ xin duyệt.
- Ưu tiên workflow dễ kiểm tra hơn "agent tự nghĩ tùy ý".

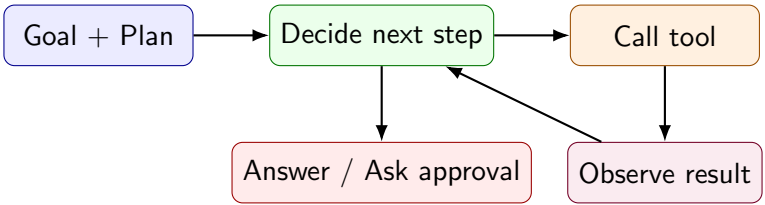
Workflow incident assistant



Thiết kế tốt

Mỗi bước có input, output, điều kiện dừng và điểm kiểm soát rõ.

Agent loop


Goal + Plan
Decide next step
Call tool
Answer / Ask approval
Observe result

Agent loop không phải chain-of-thought để lộ cho người dùng

- Không yêu cầu model hiển thị suy luận nội bộ chi tiết.
- Ta cần log hành động quan trọng: tool nào được gọi, input nào, output nào, quyết định cuối cùng.
- Có thể yêu cầu rationale ngắn ở mức kiểm toán: "vì sao gọi tool này?".
- Mục tiêu là kiểm soát workflow, không phải bắt model bộc lộ toàn bộ suy nghĩ.

Trụ cột Tool Use: tool calling là gì?

- Model không tự truy cập hệ thống; hệ thống cấp danh sách tool có schema rõ.
- Model chọn tool và sinh arguments theo schema.
- Runtime gọi tool thật, nhận kết quả rồi đưa lại cho model.
- Agent tốt phụ thuộc nhiều vào schema tool và validation.

Tool schema nên rõ thế nào?

- Tên tool mô tả đúng hành động: `get_site_kpi`, `search_runbook`.
- Input có kiểu dữ liệu, allowed values và mô tả.
- Output có cấu trúc ổn định để model đọc được.
- Tool cần trả lỗi có nghĩa: missing input, not found, permission denied.

Ví dụ tool: get_kpi()

Thành phần	Ví dụ
Mục đích	Lấy KPI gần nhất của một site
Input	site_id, time_window
Output	availability, packet_loss, latency, timestamp
Rủi ro	Dữ liệu thiếu hoặc quá cũ
Kiểm soát	Kiểm timestamp và cảnh báo nếu không đủ dữ liệu

Ví dụ tool: search_manual()

- Dùng khi agent cần tri thức từ sổ tay vận hành.
- Input nên là query và bộ lọc tài liệu nếu có.
- Output nên gồm đoạn liên quan, nguồn và phiên bản tài liệu.
- Nếu không tìm thấy bằng chứng, agent phải nói không đủ thông tin.

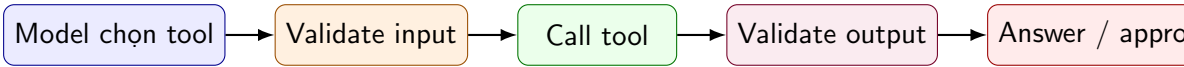
Ví dụ tool: create_recommendation()

- Tạo bản nháp đề xuất xử lý, không nhất thiết thực thi hành động.
- Output có thể gồm severity, next steps, evidence, assumptions và required approval.
- Đây là tool phù hợp để minh họa human-in-the-loop.
- Không nên để tool này tự thay đổi cấu hình thật trong lớp học.

Permission: tool nào được gọi?

- Read-only tool: ít rủi ro hơn, thường dùng trong PoC.
- Write-action tool: tạo ticket, gửi email, thay đổi cấu hình, cần duyệt.
- Admin/system tool: chỉ dùng khi có kiểm soát rất chặt.
- Nguyên tắc: least privilege – chỉ cấp đúng quyền cần cho task.

Validation input và output



Khi tool trả về lỗi

- Không để agent bịa kết quả tool.
- Nếu tool lỗi, agent cần báo lỗi hoặc yêu cầu thêm thông tin.
- Có thể retry có giới hạn, nhưng không retry vô hạn.
- Log lỗi để phân biệt lỗi tool, lỗi data và lỗi model.

Budget và timeout

- Agent có thể gọi nhiều tool nên dễ tăng chi phí và độ trễ.
- Đặt giới hạn số vòng lặp, số tool call, thời gian và token.
- Nếu vượt giới hạn, agent nên dừng an toàn và báo chưa đủ điều kiện.
- Đây là kiểm soát cần có ngay cả ở PoC.

Trụ cột Memory: dùng vừa đủ

- Với incident, memory nên lưu dữ kiện trong phiên xử lý hiện tại.
- Không tự động lưu dữ liệu nhạy cảm dài hạn nếu chưa có chính sách.
- Memory dài hạn cần cơ chế xóa, quyền truy cập và kiểm toán.
- Trong lớp học, chỉ dùng state/session memory đơn giản.

Agent không thay thế RAG

- RAG trả lời có căn cứ từ kho tài liệu.
- Agent có thể dùng RAG như một tool.
- Nếu chỉ cần hỏi-đáp tài liệu, agent có thể là quá mức cần thiết.
- Agent hữu ích khi cần nhiều bước, nhiều tool hoặc quyết định theo quy trình.

Prompt vs RAG vs Fine-tuning vs Agent

Giải pháp	Dùng khi thiếu gì?	Ví dụ
Prompt (Bài 3)	Thiếu chỉ dẫn rõ	Chuẩn hóa output JSON
RAG (Bài 5–6)	Thiếu kiến thức nội bộ/cập nhật	Hỏi sổ tay vận hành
Fine-tuning (Bài 7)	Thiếu hành vi/format ổn định ở quy mô	Phong cách hoặc nhãn chuyên biệt
Agent (Bài 8)	Thiếu workflow/tool/action	Lấy KPI + tra manual + đề xuất

Khi nào chỉ cần prompt?

- Task nằm trong năng lực sẵn có của model.
- Không cần tri thức nội bộ mới.
- Output có thể kiểm soát bằng instruction, ví dụ và schema.
- Rủi ro thấp hoặc có người duyệt kết quả.

Khi nào cần RAG?

- Câu trả lời phải dựa trên tài liệu nội bộ hoặc phiên bản cụ thể.
- Tri thức thay đổi thường xuyên.
- Cần citation/nguồn để kiểm tra.
- Không muốn model tự suy diễn ngoài context.

Khi nào cân nhắc fine-tuning?

- Cần hành vi rất ổn định trên nhiều ví dụ cùng dạng.
- Có dữ liệu chất lượng và nhãn/đầu ra chuẩn.
- Prompt quá dài hoặc chi phí inference quá cao.
- Không dùng fine-tuning để nhét tri thức thay đổi liên tục (đã học ở Bài 7).

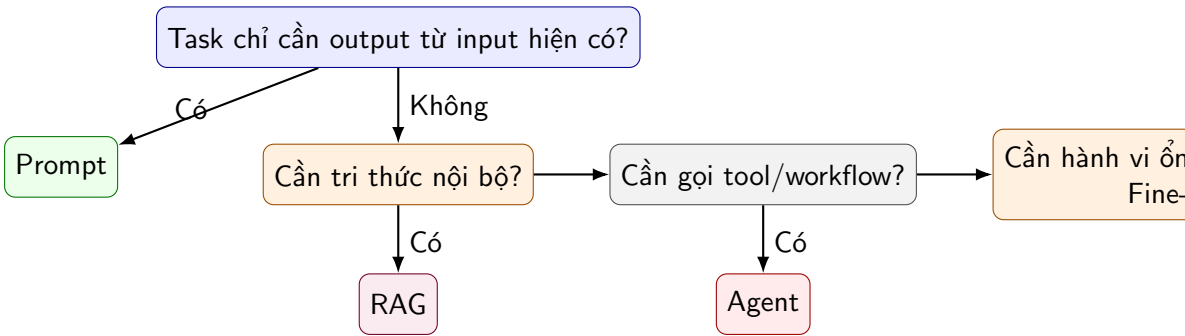
Fine-tuning: mức cần hiểu ở Bài 8

- Fine-tuning (LoRA/QLoRA) khác pre-training và khác RAG.
- Fine-tuning không tự giải quyết privacy, hallucination hay dữ liệu kém.
- Trong một agent, fine-tuning có thể cải thiện bộ quyết định nhưng không thay guardrail.
- Chi tiết thực hành đã ở Bài 7.

Khi nào cần agent?

- Task cần gọi tool hoặc API ngoài model.
- Cần nhiều bước có điều kiện.
- Cần state trong quá trình xử lý.
- Cần phân quyền và human approval cho hành động rủi ro.

Decision tree chọn giải pháp



Bảng so sánh trade-off

Giải pháp	Nhanh thử	Kiểm soát nguồn	Độ phức tạp
Prompt	Cao	Thấp	Thấp
RAG	Trung bình	Cao	Trung bình
Fine-tuning	Thấp	Không phải mục tiêu chính	Cao
Agent	Trung bình	Tùy tool/RAG	Cao

Case 1: model trả lời sai vì thiếu tài liệu

- Triệu chứng: model trả lời tự tin nhưng không đúng quy trình nội bộ.
- Không nên giải quyết bằng cách prompt dài mãi.
- Hướng phù hợp: RAG với tài liệu được kiểm soát và citation.
- Nếu còn cần gọi KPI/tool thì RAG có thể trở thành một tool trong agent.

Case 2: model biết kiến thức nhưng format không ổn định

- Triệu chứng: lúc trả JSON đúng, lúc thiếu field, lúc thêm field lạ.
- Bước đầu: prompt + schema + validation.
- Nếu quy mô lớn và có dữ liệu chuẩn: cân nhắc fine-tuning hoặc model chuyên biệt.
- Không cần agent nếu không có workflow/tool.

Case 3: cần vừa tra KPI vừa tra sổ tay

- Triệu chứng: một câu trả lời cần nhiều nguồn dữ liệu động.
- Prompt không đủ vì model không có quyền đọc KPI thật.
- RAG chỉ giải quyết phần tài liệu, không giải quyết KPI động.
- Hướng phù hợp: agent có tool `get_kpi()` và `search_manual()`.

Case 4: muốn AI tự xử lý ticket hoàn toàn

- Cần hỏi lại: ticket nào, hành động nào, hậu quả khi sai là gì?
- Nếu hành động có thể ảnh hưởng dịch vụ, không tự động hóa ngay.
- Thiết kế phù hợp: agent chuẩn bị đề xuất, người duyệt trước write-action.
- Tự động hóa chỉ sau khi có test, logging, rollback và governance.

Agent risk 1: gọi sai tool

- Agent có thể chọn tool không phù hợp với mục tiêu.
- Có thể truyền sai arguments hoặc thiếu input quan trọng.
- Kiểm soát: schema rõ, validation, allowed tools theo role, hỏi lại khi thiếu dữ liệu.
- Không cho agent tự suy diễn input quan trọng.

Agent risk 2: vòng lặp và chi phí

- Agent có thể lặp tool call mà không tiến triển.
- Chi phí token, API và thời gian tăng nhanh.
- Kiểm soát: max steps, timeout, budget và điều kiện dừng.
- Khi dừng, agent cần trả trạng thái "chưa đủ điều kiện", không bịa kết luận.

Agent risk 3: quyền vượt mức

- Tool có thể mở rộng quyền của model sang hệ thống thật.
- Rủi ro cao nếu tool có quyền ghi, xóa, gửi hoặc thay đổi cấu hình.
- Kiểm soát: least privilege, scoped token, approval, audit log.
- Phân biệt read action và write action ngay trong thiết kế.

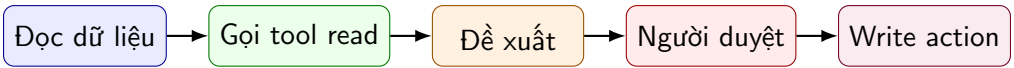
Agent risk 4: dữ liệu nhạy cảm

- Agent gom dữ liệu từ nhiều nguồn, nên rủi ro privacy tăng.
- Không đưa thông tin nhạy cảm vào model/service nếu chưa được phép.
- Cần mask/redact dữ liệu khi có thể.
- Log cũng có thể chứa dữ liệu nhạy cảm, không chỉ input ban đầu.

Agent risk 5: hành động không thể đảo ngược

- Một số hành động có hậu quả lớn: thay đổi cấu hình, xóa dữ liệu, gửi thông báo chính thức.
- Với hành động này, agent chỉ nên chuẩn bị bản nháp hoặc đề xuất.
- Người có thẩm quyền duyệt trước khi thực thi.
- Cần cơ chế rollback nếu tự động hóa trong tương lai.

Human-in-the-loop đặt ở đâu?



Nguyên tắc

Đặt người duyệt trước hành động có rủi ro cao hoặc khó đảo ngược.

Guardrail tối thiểu cho agent PoC

- Danh sách tool được phép gọi.
- Schema input/output và validation.
- Max steps, timeout, budget.
- Logging tool call và kết quả.
- Rule "không đủ dữ liệu thì không hành động".
- Human approval trước write-action.

Metric đánh giá agent

- Task success: hoàn thành đúng mục tiêu không?
- Tool correctness: gọi đúng tool và đúng input không?
- Grounding: kết luận có dựa trên dữ liệu/tool result không?
- Safety: có tránh hành động rủi ro khi chưa duyệt không?
- Cost/latency: có nằm trong ngân sách không?

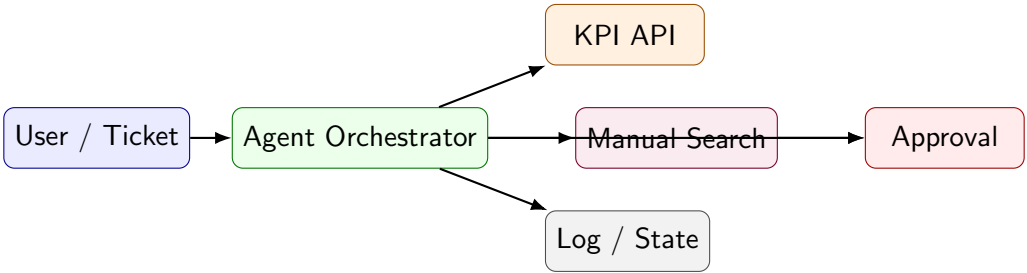
Error analysis cho agent

Loại lỗi	Dấu hiệu	Cách xử lý
Goal mơ hồ	Gọi tool lan man	Viết lại task
Tool sai	Output không liên quan	Giới hạn allowed tools
Input sai	Tool trả lỗi	Validate và hỏi lại
Kết luận sai	Không theo evidence	Bắt cite tool result
Rủi ro cao	Tự write-action	Human approval

Mini design: incident agent

- Input: mô tả incident, site_id, thời gian.
- Tools: get_kpi, search_manual, create_recommendation.
- Output: tóm tắt, bằng chứng, giả thuyết, bước kiểm tra, mức cần duyệt.
- Guardrail: nếu thiếu KPI hoặc manual evidence thì không kết luận nguyên nhân chắc chắn.

Sơ đồ kiến trúc PoC



Câu hỏi thiết kế trước khi xây agent

1. Agent được phép đọc gì?
2. Agent được phép ghi hoặc thay đổi gì?
3. Tool nào là read-only, tool nào là write-action?
4. Khi thiếu dữ liệu, agent phải dừng hay hỏi lại?
5. Ai duyệt hành động rủi ro?

Checklist trước khi gọi một hệ thống là agent

- Có goal rõ (Planning).
- Có tool schema rõ (Tool Use).
- Có state/log để kiểm tra luồng xử lý (Memory).
- Có validation và giới hạn vòng lặp.
- Có human approval trước hành động rủi ro.
- Có test case và failure case.

Knowledge check 1

Tình huống

Model phân loại ticket chưa ổn định nhưng không cần gọi tool hay tra tài liệu mới.

- Nên thử gì trước: Agent, RAG, prompt/schema hay fine-tuning?
- Cần test set như thế nào?
- Khi nào mới cân nhắc fine-tuning?

Knowledge check 2

Tình huống

Kỹ thuật viên hỏi: “Site HN-023 có nên reboot thiết bị không?” Cần KPI hiện tại và quy trình nội bộ.

- Prompt đơn thuần có đủ không?
- RAG giải quyết phần nào?
- Agent cần tool nào và điểm duyệt ở đâu?

Knowledge check 3

Tình huống

Agent gọi `get_kpi()` nhưng dữ liệu trả về cũ hơn 24 giờ.

- Agent có nên kết luận nguyên nhân không?
- Guardrail nên viết thế nào?
- Output cho người dùng nên nói gì?

Những hiểu nhầm thường gặp

- "Agent càng tự động càng tốt." Sai: mức tự động phụ thuộc rủi ro.
- "RAG và agent là một." Sai: RAG có thể là một tool trong agent.
- "Fine-tuning giúp model biết tài liệu mới." Không phù hợp với tri thức thay đổi thường xuyên.
- "Có tool calling là an toàn." Sai: tool calling cần permission, validation và approval.

Kết nối sang Lab

- Lab sẽ dùng ba tool giả lập: KPI, knowledge, recommendation.
- Mục tiêu: thấy rõ Planning (workflow), Tool Use (gọi tool), Memory (state) và điểm kiểm soát.
- Học viên sẽ thêm rule "không đủ dữ liệu thì không hành động".
- Học viên sẽ thêm human approval trước write-action giả lập.

Tổng kết Bài 8 và Module 3

- Agent = LLM + **Planning** + **Memory** + **Tool Use**, đặt trong workflow có control.
- Dùng prompt khi thiếu chỉ dẫn; RAG khi thiếu tri thức; fine-tuning khi thiếu hành vi ổn định; agent khi thiếu workflow/tool/action.
- Rủi ro của agent cao hơn chatbot vì agent có thể dùng tool và tạo hành động.
- Human-in-the-loop là một phần của thiết kế, không phải phần bổ sung sau cùng.

Khép lại Module 3

Tổng quan/Transformer → Responsible AI → Prompt/API/HF → CV → RAG pipeline & Vector DB → RAG end-to-end → Fine-tuning/NLP → Agent: luôn bắt đầu từ nghiệp vụ, đo được và kiểm soát được.

Exit ticket

1. Nêu một use case chỉ cần prompt, không cần agent.
2. Nêu một use case cần RAG nhưng chưa cần agent.
3. Nêu một use case cần agent và chỉ ra ít nhất một điểm human approval (Planning + Tool Use + Memory).